

Air University



COAL

Year 2025

Ms. Sonia Shehzadi

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: December 23, 2025

Lab Task # 14

Question

Code

```
.model small
.386                ; Required for 32-bit registers or advanced instructions if
needed
.stack 100h

.data
; Declare two 16-bit values for comparison
source    DW 500
target    DW 300

; Output messages
msg_above DB 'Source is greater than Target.$'
msg_below DB 'Source is not greater than Target.$'

.code
Main PROC
; Initialize Data Segment
    mov     ax, @data
    mov     ds, ax

; Ensure correct setup of the ES segment for string instructions
    mov     es, ax

; Load the addresses of source and target into SI and DI
    lea     si, source
    lea     di, target

; CMPSW compares the word at DS:SI with the word at ES:DI
    cld
    cmpsw

; Use a conditional jump (JA - Jump if Above/Unsigned Greater)
    ja      SourceIsGreater

; --- ELSE: Source <= Target ---
    lea     dx, msg_below
    jmp     DisplayMessage

SourceIsGreater:
; --- IF: Source > Target ---
    lea     dx, msg_above

DisplayMessage:
; Display the appropriate message using INT 21h (function 09h)
    mov     ah, 09h
    int     21h
```

```
    ; Terminate Program
        mov     ah, 4ch
        int     21h

Main ENDP
end Main
```

Output

```
D:\>D:\ntest
Source is greater than Target.
D:\>_
```

Explanation:

Explanation of the CMPSW logic in 5 points:

1. **Segment Initialization:** String instructions require the **ES (Extra Segment)** to be initialized. In this code, ES is set to the same address as DS (mov es, ax) so that the program looks for both the source and target variables in the same data segment.
2. **Pointer Setup:** The LEA (Load Effective Address) instruction is used to load the memory address of the source into the **SI (Source Index)** register and the target into the **DI (Destination Index)** register.
3. **The CMPSW Instruction:** The CMPSW (Compare String Word) instruction subtracts the 16-bit word at ES:DI from the word at DS:SI. It does not change the values in memory; it only updates the **CPU Status Flags** (like the Zero and Carry flags) based on the result.
4. **Direction and Increments:** By using CLD (Clear Direction Flag), the program ensures that SI and DI are automatically incremented by 2 after the comparison. This "points" the registers to the next word in memory, preparing them for any further string operations.
5. **Conditional Branching:** The JA (Jump if Above) instruction checks the status flags set by CMPSW. If the value in source is unsigned-greater than the value in target, the code jumps to the SourceIsGreater label; otherwise, it continues to the next line to handle the "less than or equal" case.

THE END